병원 동행 매칭 서비스

오늘 병원,
누구랑 가지?

믿을 수 있는 동행 매니저와 함께,
병원 가는 길부터 진료까지 안전하게 동행합니다.

Team06 · 훈희는 못말려
최훈희 · 김신영 · 나희원 · 박현호 · 장윤찬

멘토 김동기 | 2026.09.04 – 10.01
프로그래머스 데브코스 백엔드 14회차 2차 팀 프로젝트



CONTENTS

목차

가지?

01 프로젝트 개요 — 주제 · 차별점 · 기술 · 구조

02 팀 구성 및 역할 — 팀원 5명 · 멘토

03 수행 절차 및 방법 — 절차 · 주차별 일정

04 수행 경과 — 핵심 기능 · 상태 관리 · 인증 · 테스트

05 자체 평가 의견 — 완성도 · 개선점 · 회고

01

프로젝트 개요

주제 및 선정 배경

차별화 포인트

서비스 흐름

활용 장비 및 재료

프로젝트 구조

활용 방안 및 기대 효과

주제 및 선정 배경

왜 만들었나

병원이 없어서가 아니라

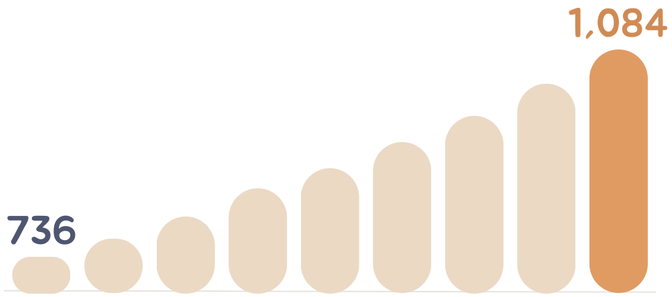
데려다줄 사람이 없다

어르신 혼자는 어렵다
낙상 · 길찾기 · 수납 · 진료 안내

자녀는 매번 연차를 쓴다
평일 진료에 맞추려면 반차가 필요

1 고령자는 계속 증가

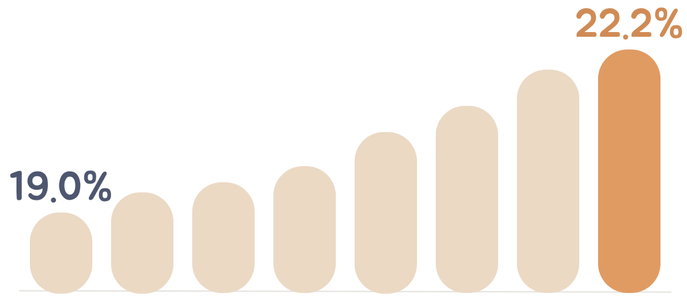
전국 65세 이상 인구 추이 · 만 명



8년 만에 736만 → 1,084만 명.
해마다 약 5%씩 늘고 있다.

2 혼자 사는 비율도 증가

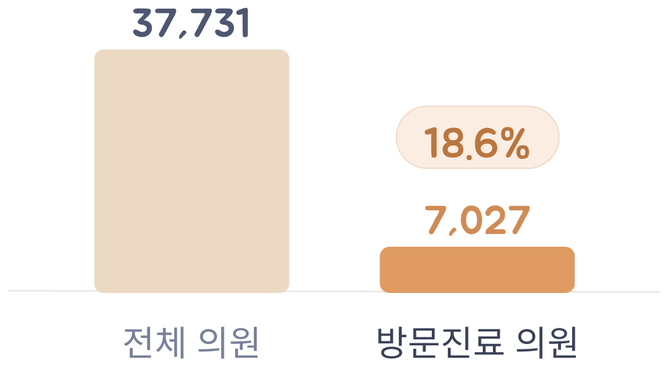
독거노인 비율 · %



노인 다섯 중 한 명은 혼자 산다.
같이 병원 갈 사람이 곁에 없다.

3 방문 진료는 18.6%뿐

전체 의원 대비 방문진료 의원 수 · 개



거동이 불편한 환자 중 실제 이용은
8.4%(23,274명)에 그친다.

출처 · 통계청 주요 인구 지표 · 국가데이터처 「장래가구추계·장래인구추계 2022」(2024) · 건강보험심사평가원 · 2022년 보건행정학회지

차별화 포인트

무엇이 다른가

01

	케어○○○ 간병 중심 플랫폼	위드○○○ 전문 인력 배정	가지? 우리 서비스
매칭 방식	입찰제	전문가 배정	**실시간 매칭**
인력 풀	제한 없음	협업 센터 인력	**교육영상을 수료한 이수자 누구나**
최소 이용	3시간	—	**제한 없음**
요금	13,000~18,000원/시간 수수료 12% + 별도 부과세 10%	15,000~25,000원/시간	**의뢰인이 시급을 산정**
진료 기록	동행인이 직접 작성	동행인이 직접 작성	**동행인이 간단하게 작성 + AI 요약 제공**

프로젝트 내용

서비스 흐름

01 의뢰인

📝 **공고 등록**

병원 · 날짜 · 예상 시간을
적어 공고를 올립니다.

→

02 의뢰인

💳 **선결제**

예정 시간 기준 금액을
선결제해야 공고가 등록됩니다.

→

03 동행인

🙋 **동행인 지원**

교육영상을 이수한 동행인만
지원할 수 있습니다.

→

04 의뢰인

🤝 **매칭 확정**

지원자 한 명을 승인하면
공고가 닫힙니다.

→

05 동행인

🚕 **동행 진행**

출발 → 병원 이동 →
병원 도착 → 귀가 완료
순서로만 넘어갑니다.

→

06 동행인

📄 **진료 보고서(선택)**

병원 이동부터 기록하고
AI가 내용을 요약합니다.

→

07 양쪽

💰 **리뷰와 정산**

실제 시간으로 금액 확정,
차액은 결제나 환불로 진행됩니다.

누가 움직이나

의뢰인 공고 · 결제 · 승인

동행인 지원 · 기록 · 보고서

양쪽 정산 결과 확인

활용 장비 및 재료

기술 스택

Backend

Java 25
Spring Boot 4.1.1
Spring Data JPA

Security

Spring Security
JWT (jjwt 0.13.0)
Refresh Token

Database

MySQL 8.0
H2 (test)
Docker

Frontend

Next.js 16
React 19 · TypeScript
Tailwind CSS 4

External

토스페이먼츠
Claude API
카카오맵

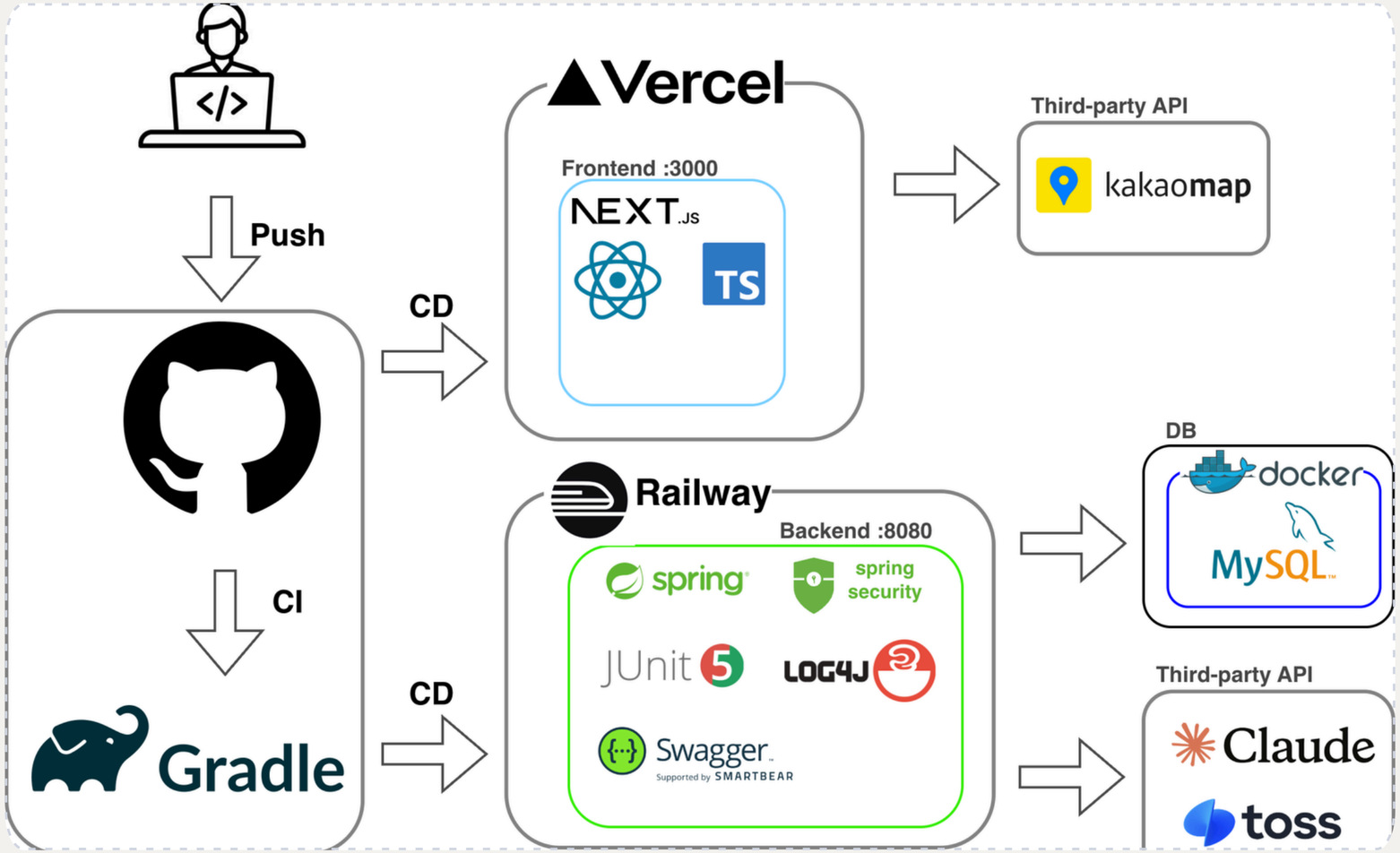
Infra · Docs

GitHub Actions · Vercel
Swagger UI · Notion
devcproject.cloud

프로젝트 구조

시스템 구조

01



클라이언트
Next.js

서버
Spring Boot · 11개 도메인

데이터
MySQL

외부
토스 · Claude · 카카오맵

활용 방안 및 기대 효과

무엇이 달라지나

의뢰인

연차를 쓰지 않아도
어르신이 안전하게
병원에 다녀옵니다

보호자

진료 내용을 요약으로 확인
회사에 있어도
상태를 파악합니다

동행인

하루 몇 시간짜리 일자리
필요한 시간만큼
유연하게 참여합니다

신뢰

누적 동행 기록과 등급 배지
교육 이수를 검증한
동행인만 지원합니다

사회

돌봄 공백을
하루 몇 시간짜리
작은 일자리로 메웁니다

확장

요양기관 제휴
정기 진료 구독
지자체 연계

TEAM

팀 구성 및 역할

멘토 김동기



최훈희

팀장

회원 · 인증 · 인가
교육 영상 시청 검증
ERD 설계



김신영

팀원

공고 도메인
카카오맵 연동
스케쥴러
로깅 · 모니터링



나희원

팀원

지원 도메인
동행 진행 상태
와이어프레임



박현호

팀원

결제 · 정산 · 이동수단
정산 스케줄러
회의 · 멘토링 기록



장윤찬

팀원

진료 보고서 · 리뷰
AI 요약 연동
전체 통합 테스트



공통

담당 도메인 동시성 테스트 · 프론트 화면 연동



멘토 김동기

주제 선정 피드백, 프로젝트 질의응답

PROCESS

수행 절차 및 방법



기획

주제 선정



설계

ERD · API



구현

도메인 분담



테스트

실패 테스트 구현



통합

PR 리뷰



검증

전체 테스트



배포

Vercel



브랜치 전략

feat → dev → main

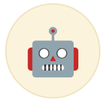
기능 단위로 브랜치를 따고
dev 에 모은 뒤 main 으로 올립니다.



코드 리뷰

PR 단위 상호 리뷰

머지된 PR 142건을 서로 읽고
승인해야 dev 에 들어갑니다.



CI

GitHub Actions

푸시할 때마다 자동으로 빌드하고
테스트를 돌립니다.



테스트 우선

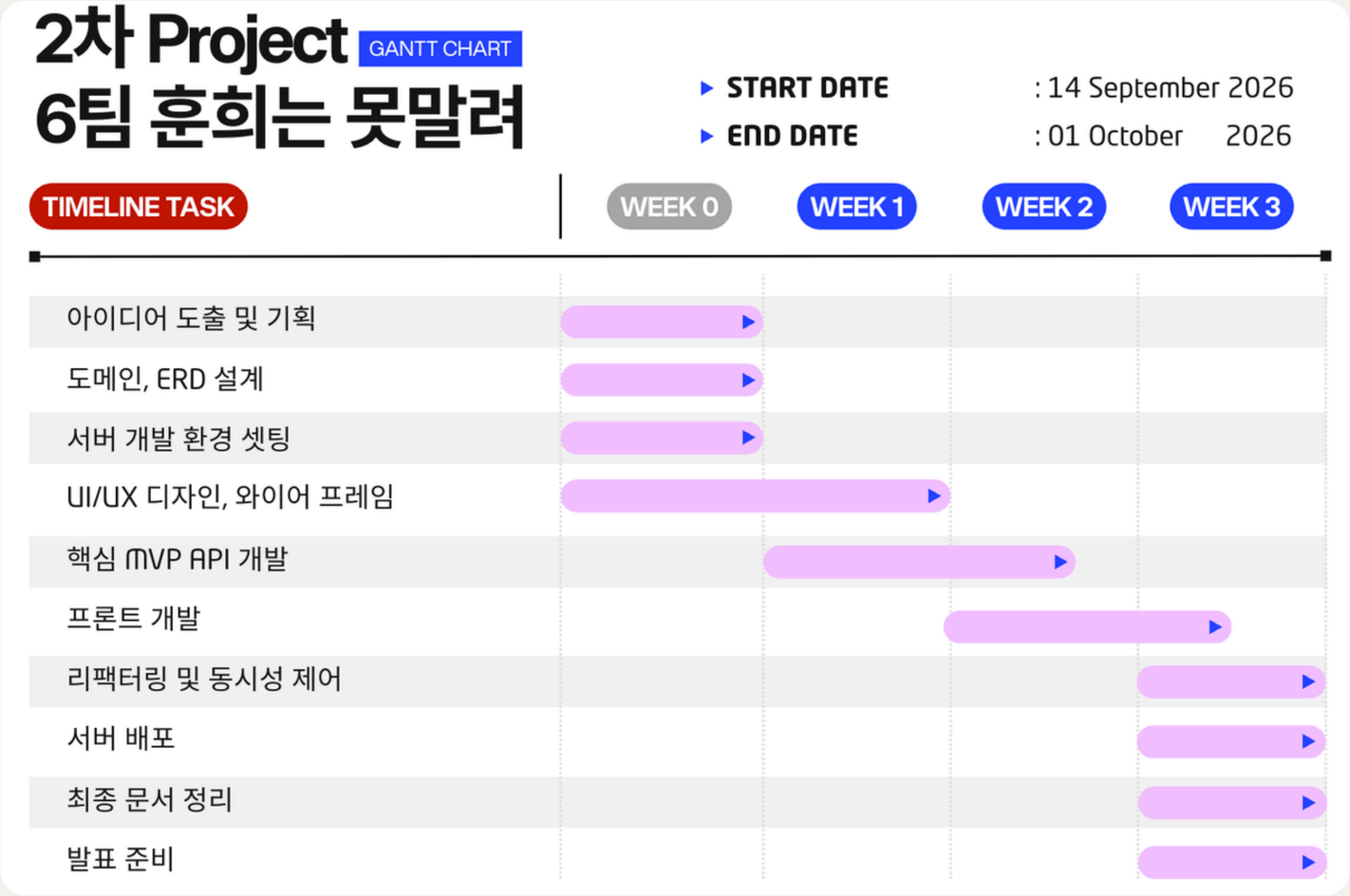
실패하는 테스트부터

동시성 버그 7건은 모두 재현
테스트를 먼저 쓰고 고쳤습니다.

TIMELINE

간트차트를 활용한 주차별 일정

2026.09.04 – 10.01



672

커밋

142

Pull Request

240

테스트 케이스

04

프로젝트 수행 경과

핵심 기능 | 상태값 관리 | 하트비트 | 인증 · 인가 | 동시성 | 테스트 | 시연

핵심 기능

공고와 매칭



모집 중

OPEN

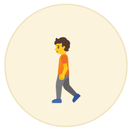
→



매칭 완료

MATCHED

→



동행 중

IN_PROGRESS

→



동행 완료

COMPLETED



한 명을 승인하면

같은 공고의 나머지 지원은 자동 거절됩니다.

```
// 같은 공고의 나머지 지원자 자동 거절
samePostPendingApplications.stream()
  .filter(other → !other.getId().equals(applicationId))
  .forEach(Application::reject);
```



시간이 겹치면

같은 동행인의 다른 지원은 불가합니다.

```
// 두 동행 시간이 겹치면 승인 자체를 막는다
return firstPost.getEscortStartAt().isBefore(secondPost.getEscortEndAt())
      && firstPost.getEscortEndAt().isAfter(secondPost.getEscortStartAt());
```



모집 마감 시간이 지나면

1분마다 도는 스케줄러가 공고를 만료 처리합니다.

```
@Scheduled(fixedRate = 60000)   // 1분마다 실행
public void expireOverduePosts() {
  postService.expireOverduePosts();
}
```

핵심 기능

LLM기술을 활용한 AI 보고서

AI 요약 파이프라인

원문 저장 → 마스킹 → 프롬프트 → 응답 검증 → 요약 저장

프롬프트로 환각을 막는다

5개 필드 JSON 스키마 + Few-shot 예시 1개
원문에 없으면 null, 진단·의료 조언은 금지

LLM 응답은 검증하고 저장한다

text 블록만 모으고 { } 구간만 추출
모르는 필드는 무시해 파싱 실패 방지

실패해도 원문은 지킨다

트랜잭션 밖 호출 · 타임아웃, 실패 시 '요약 대기'로 보존

동행인이 남기는 진행 기록

병원 이동부터 보고서 작성

출발 → 병원 이동 → 병원 도착 → 귀가 중 → 귀가 완료



핵심 기능

결제와 정산

04



공고 등록 시 선결제

예정 시간 기준

→



동행 완료

실제 시간 확정

→



금액 조정

추가 결제 / 부분 취소

→



정산

평일 오전 10시

10%

플랫폼 수수료

10%

노쇼 패널티 차감

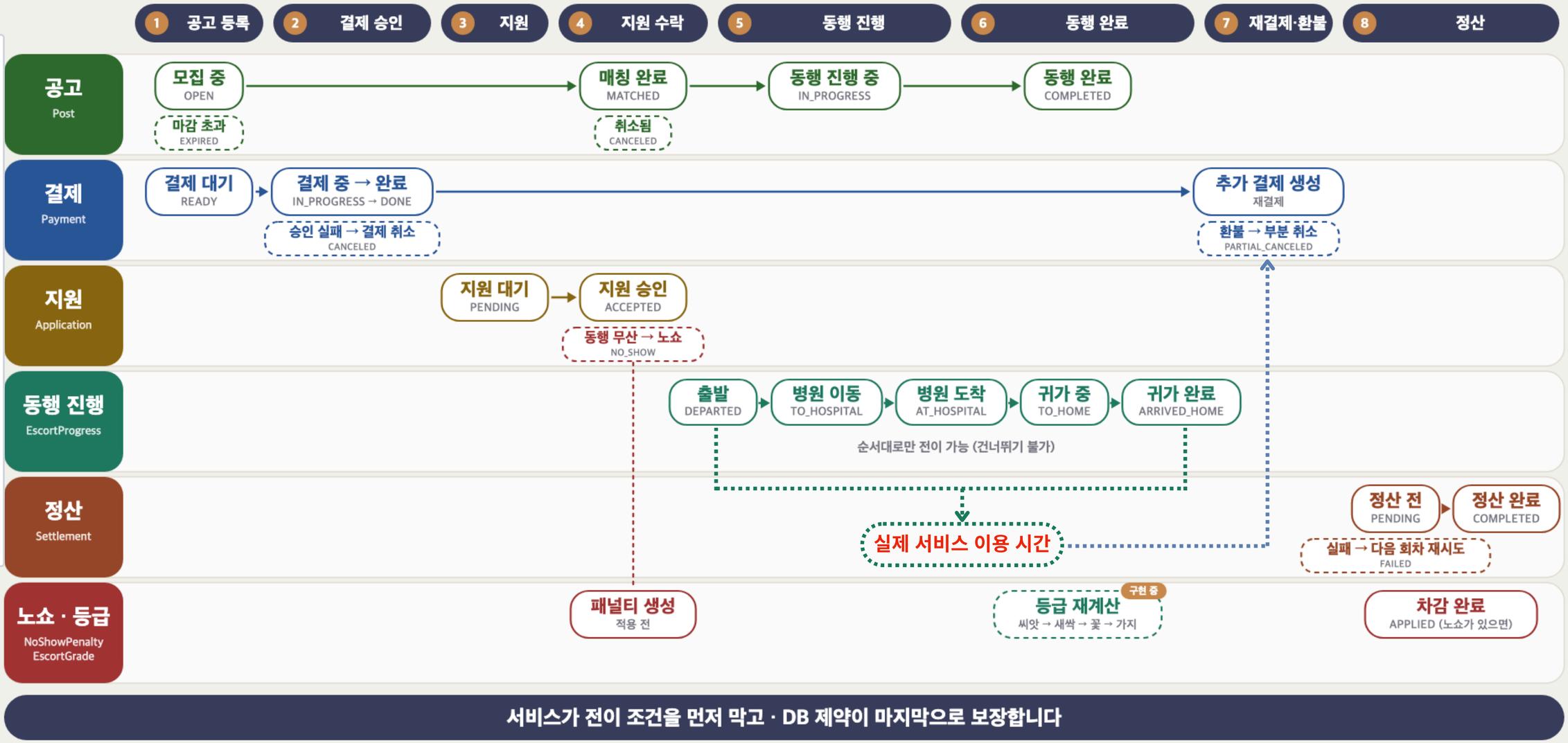


실패 건 자동 재시도

다음 정산 회차에 다시 처리합니다.

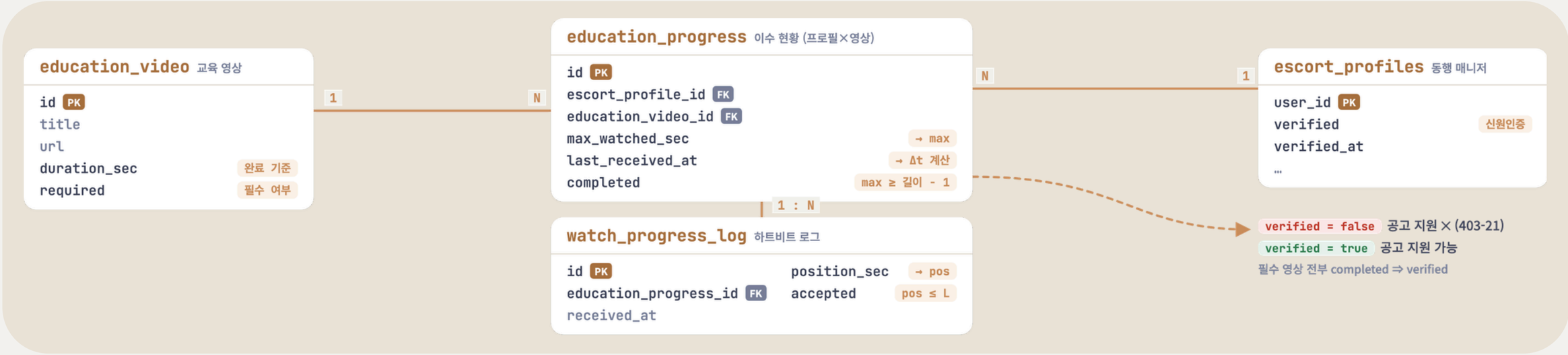
정확성

한 건의 동행이 거치는 상태값



핵심 기능

하트비트 기술을 활용한 교육영상 시청관리



$$L = \underline{max} + \underline{min(\Delta t,\ 15)} \times \underline{1.2} + \underline{2}$$

max	min(Δt, 15)	× 1.2	+ 2
지금까지 인정된 위치 (maxWatchedSec)	Δt = 직전 요청 후 경과 시간 15초 상한 · 공백 점프 차단	재생 속도·타이머 오차 경과 시간의 20% 여유	고정 여유 2초 첫 요청(Δt=0) · 네트워크 지연

L = 이번 요청의 허용 한도 | pos ≤ L → 인정 (max ← pos) | pos > L → 거부, max로 되돌림

인증 · 인가

누가(인증) 무엇을 할 수(인가) 있는가

토큰 흐름

클라이언트

Access T

401-111 Unauthorized

재발급 요청 · Refresh 쿠키 자동 전송

새 Access Token 재발급

```
.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")    // 관리자
.requestMatchers(HttpMethod.POST, "/api/v1/users").permitAll()  // 회원가입
.requestMatchers(HttpMethod.GET, "/api/v1/users/username").permitAll()  // username 중복 검사
.requestMatchers(HttpMethod.GET, "/api/v1/users/email").permitAll()  // email 중복 검사
.requestMatchers(HttpMethod.POST, "/api/v1/auth/login").permitAll()  // 로그인
.requestMatchers(HttpMethod.POST, "/api/v1/auth/refresh").permitAll()   // access 토큰 재발급
// swagger
.requestMatchers("/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs/**").permitAll()
// 프로필
.requestMatchers(HttpMethod.GET, "/api/v1/users/*/profile/client").hasAnyRole("ADMIN", "ESCORT")
.requestMatchers("/api/v1/users/profile/client").hasRole("CLIENT") // 의뢰인 프로필 생성, 조회, 수정
.requestMatchers("/api/v1/users/profile/escort").hasRole("ESCORT") // 동행인 프로필 생성, 조회, 수정
.requestMatchers("/api/v1/users/*/profile/escort").hasAnyRole("CLIENT")    // 동행인 프로필 타인 조회
```

회원 관리 · 공고 관리

토큰 폐기

로그아웃 · 토큰 폐기 + 쿠키 삭제
회원 탈퇴 · 계정의 모든 토큰 폐기

기술적 도전 ①

동시에 요청이 몰릴 때

재현한 경쟁 상태 7건

실패하는 테스트로 먼저 재현한 뒤 고쳤습니다

중복 지원 · 동시 승인 · 평점 유실 · 보고서 중복 · 취소 겹침 · 정산 중복 · 프로필 덮어씀

🔒 비관적 락

먼저 들어온 요청이 그 행을 잠그고,
뒤에 온 요청은 줄을 서서 기다립니다.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT p FROM Post p WHERE p.id = :id")
Optional<Post> findByIdWithLock(@Param("id") Long id);
```

같은 공고를 동시에 승인 · 취소 경로

➕ 원자적 벌크 UPDATE

읽고 더해서 쓰지 않고,
DB 가 직접 더하게 맡깁니다.

```
@Modifying(flushAutomatically = true, clearAutomatically = true)
@Query("""
 update EscortProfile e
 set e.ratingSum = e.ratingSum + :rating,
  e.ratingCount = e.ratingCount + 1,
  e.version = e.version + 1
 where e.userId = :userId
 """)
int addRating(@Param("userId") Long userId, @Param("rating") int
rating);
```

리뷰 평점 유실 · 완료 건수 누락

🚦 조건부 UPDATE 로 선점

UPDATE 가 바꾼 행이 1이면 내가 잡은 것,
0이면 남이 먼저 가져간 것입니다.

```
@Modifying
@Query("""
    update Settlement s
    set s.settlementStatus = PROCESSING
    where s.id = :settlementId
    and (s.settlementStatus = PENDING
    or s.settlementStatus = FAILED)
""")
int updateProcessing(Long settlementId);
```

정산이 두 번 나갈 뻔한 상황

기술적 도전 ②

팀원 트러블슈팅

⏱️ 기획관점의 트러블슈팅

최종 결제 금액의 기준 시각

문제 결제를 미뤄도 불이익이 없다

원인 금액이 귀가 시각으로 확정된다

두 안을 놓고 회의에서 결정

결제 시각 기준

미룰수록 금액이 늘어
빠른 결제를 유도한다

현재 식 유지 ✓

실제 이용 시간만큼만
의뢰인이 결제한다

대신 공고 등록 시 선결제로 동행인의 최소 보장을 남겼다

🧾 기술관점의 트러블슈팅

결제 승인의 트랜잭션 경계

문제 호출 전 예외도 취소로 처리됐다

원인 컨트롤러가 모든 예외를 잡았다

한 덩어리를 세 단계로 분리

1 **검증** 금액 확인 → 2 **외부 호출** 토스 승인 → 3 **DB 반영** 별도 빈 · 여기서만 취소

같은 클래스 안에서 부르면 프록시를 안 타
@Transactional 이 안 먹는다

MENTOR FEEDBACK

멘토 피드백과 반영

멘토 김동기

1 동시성 검증 방식

멘토 ExecutorService · CountDownLatch 로 검증하라

→ **반영** 스레드를 동시에 출발시켜 경쟁 상태 4건을 재현

반영 완료

2 DTO 설계

멘토 등록과 수정 DTO 를 처음부터 분리하라

→ **반영** 프로필 수정 DTO 를 등록용과 분리

반영 완료

3 보고서 저장 형식

멘토 TEXT 말고 JSON 으로 받아 형식을 맞춰라

→ **반영** AI 요약을 JSON 으로 저장, 서버에서 형식 지정

반영 완료

4 남은 과제

멘토 예외 코드는 ENUM, 삭제는 상태값으로

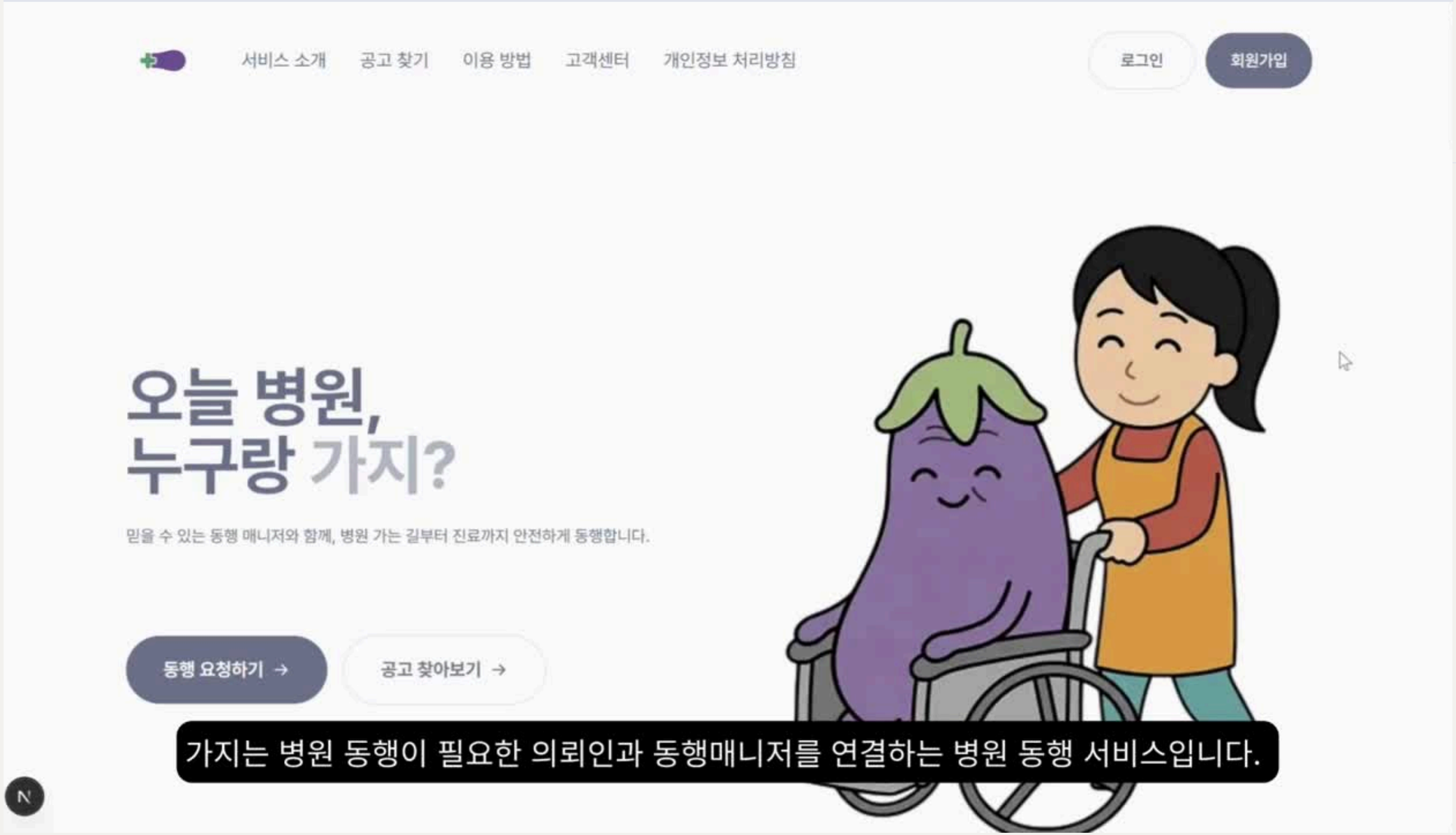
→ **반영** 3차 프로젝트에서 반영 예정

예정

DEMO

시연

devcproject.cloud



05

자체 평가 의견

완성도

잘한 점과 아쉬운 점

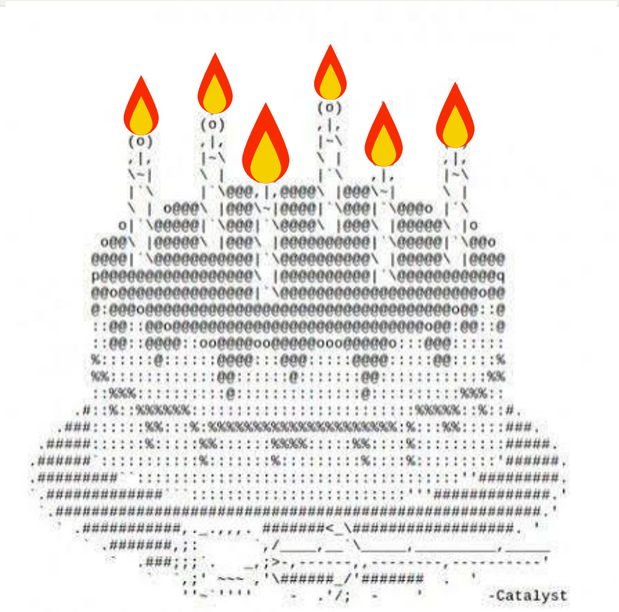
추후 개선점

느낀 점

REVIEW

자체 평가

10점 만점



완성도

8 / 10

잘한 점

1. 팀원 모두 적극적으로 참여
2. 역할을 분담하고 맡은 업무를 끝까지 수행
3. 어려운 프로젝트였지만 핵심 기능 구현 완료

아쉬운 점

1. 제한된 일정으로 일부기능은 추가하지 못함
2. 처음 설계단계부터 동시성을 고려하지 못함

NEXT

추후 개선점

발견한 과제

1

실시간 위치를 실좌표로

지금은 목업 좌표를 쓰고 있어
쓰기가 잦아 저장 방식부터 다시

2

회원 개인정보 암호화

회원 정보를 암호화하여 저장, 관리

3

AI 설정을 yaml 로

baseUrl · 헤더를 yaml 로 빼고
모델 버전은 상수로 고정

4

테스트 DB 일치

H2 대신 실제 MySQL 로 돌려
환경 차이를 없애기

5

취약점 점검

SQL 인젝션 · 토큰 탈취 대비를
테스트에 포함하기

6

성능 테스트

실사용자 규모의 응답 시간 측정
3차 프로젝트에서 진행

RETROSPECTIVE

느낀 점

팀원 5명

최훈희

인증 · 인가 기능 구현을 배웠고, 개발 초반부터 동시성은 미리 생각해두고 만들어야 한다고 느꼈습니다.
나중에 손대려니 고쳐야 할 곳이 너무 많았습니다.

김신영

분석설계부터 배포 테스트까지 팀원이 깃허브를 통해 함께 구현해 나가면서 팀프로젝트경험을 쌓을 수 있었습니다.

나희원

기획부터 자료조사, API 명세까지 직접 해봤습니다.
다른 프로젝트에서는 챙기지 못했던 동시성을 이번에는 챙길 수 있었습니다.

박현호

팀원 다섯 명이 각자 도메인을 맡아 하나로 합쳐봤습니다.
혼자서는 배울 수 없는 것들을 배운 3주였습니다.

장윤찬

팀원들과 하나의 서비스를 완성해가며 협업과 문제 해결의 중요성을 배웠습니다.



오늘 병원, 누구랑 가지?

devcproject.cloud

Team06 · 훈희는 못말려

감사합니다

질의응답